
Continuous Trajectory Optimization with Machine Learning Techniques

Lirui Wang

University of Washington

Seattle, USA

`liruiw@cs.washington.edu`

Abstract

1 Trajectory optimization-based motion planning in robotics aims to find a feasible,
2 optimal trajectory between two fixed end points. Feasibility evaluates task-specific
3 constraints such as collision and joint limits, and optimality often requires the
4 trajectory to be smooth. A drawback of traditional optimization methods is that
5 they require a fine discretization of the trajectory to carefully reason about cost
6 information. In this work, we investigate how machine learning techniques can
7 be applied in trajectory optimization-based motion planners. Specifically, we
8 will discuss the ideas of Marinho et al. [2016] and Mukadam et al. [2018] that
9 enable efficient continuous trajectory optimization. While Marinho et al. [2016]
10 introduce functional gradient descent trajectory optimization algorithm for robot
11 motion planning in Reproducing Kernel Hilbert Spaces (RKHSs), Mukadam et al.
12 [2018] formulate motion planning as probabilistic inference problem in Gaussian
13 Processes (GPs).

1 Introduction and Related Works

15 Trajectory optimization starts with a possibly infeasible trajectory and then optimizes the trajectory
16 by minimizing a cost function. CHOMP Ratliff et al. [2009] and related methods Dragan et al.
17 [2011] optimize a cost functional using covariant gradient descent, while STOMP Kalakrishnan
18 et al. [2011] uses stochastic sampling of noisy trajectories to optimize non-differentiable costs.
19 More recently, TrajOpt Schulman et al. [2014] solves a sequential quadratic program and performs
20 convex continuous-time collision checking. While these optimization-based methods have shown
21 improvement on planning time and performance, in practice a fine discretization of the trajectory is
22 necessary to integrate cost information when reasoning about small obstacles.

23 A continuous-time trajectory representation can avoid some of these challenges and accelerate the
24 planning time. Machine learning ideas provide a compact representation to handle continuous-time
25 trajectory. In this work, we summarize the contributions of Marinho et al. [2016] and Mukadam
26 et al. [2018] and provide some insights and discussions of their ideas. Marinho et al. [2016] RKHS
27 can provide natural smoothness in function approximation and reduce the number of waypoints
28 in parametrization. They propose to minimize the cost functional regularized by the RKHS norm.
29 Smooth trajectories are represented as vectors in an RKHS and different kernels can lead to different
30 notions of smoothness. A functional gradient trajectory optimization method based on RKHS
31 representation is introduced and demonstrated to optimize efficiently through taking larger steps than
32 waypoint parametrization.

33 On the other hand, Mukadam et al. [2018] GPMP2 models planning as a inference problem, which
34 captures optimality by a prior on trajectories and encodes feasibility in a likelihood function. The
35 Bayesian inference idea allows them to perform efficient inference on factor graphs by solving least
36 square problems. Using GP interpolation, the trajectory can be queried at any point of interest.

Moreover, their simulation and real experiments demonstrate that the methods not only demonstrates high success rate but also improves the planning time dramatically.

2 Preliminaries

2.1 Trajectory Optimization

A trajectory is a function $\xi : [0, 1] \mapsto \mathcal{C}$ that maps time $t \in [0, 1]$ to robot configuration space $\mathcal{C} \subset \mathbb{R}^d$. $\xi(t)$ defines the robot configuration at time t . The goal of motion planning is to find trajectory that satisfies constraints and minimizes costs, so we can formulate the objective as

$$\min \mathcal{F}(\xi(t))$$

where the functional $\mathcal{F}$ often contains measure of optimality such as smoothness in terms of velocity, acceleration, and jerks etc. and measure of feasibility like proximity to nearest obstacles. For the purpose of this review, we ignore trajectory constraints such as joint limits.

2.2 Reproducing Kernel Hilbert Space

For a compact subset $\mathcal{X} \subset \mathbb{R}^d$ and a Hilbert space $\mathcal{H}$ of functions f , we say $\mathcal{H}$ is an RKHS if there exists K such that K 's span is dense in $\mathcal{H}$ and K is a reproducing kernel. We say $K(x, y)$ is a reproducing kernel of a Hilbert space $\mathcal{H}$ if for any $f \in \mathcal{H}$, we have

$$f(x) = \langle K(x, \cdot), f(\cdot) \rangle_{\mathcal{H}} = \langle f(\cdot), K(\cdot, x) \rangle_{\mathcal{H}} \quad (1)$$

An example of K could be Dirac delta function $\delta_y(x)$. Let g and $h \in \mathcal{H}$ be:

$$g(\cdot) = \sum_{i=1}^M \alpha_i K(\cdot, x_i), f(\cdot) = \sum_{j=1}^N \beta_j K(\cdot, x_j) \quad (2)$$

We define the inner product to be

$$\langle g, h \rangle_{\mathcal{H}} = \sum_{i,j} \alpha_i \beta_j K(x_i, x_j) \quad (3)$$

2.3 Gaussian Process

A random process $(X_t)_{t \in T}$ is called a Gaussian process if for any finite subset $T_0 \subset T$, the random vector $(X_t)_{t \in T_0}$ has a normal distribution. Using the GP framework, we can say that for any collection of times $t = \{t_0, \dots, t_N\}$, ξ has a joint Gaussian distribution:

$$\xi = [\xi_0, \dots, \xi_N] \sim \mathcal{N}(\mu, K) \quad (4)$$

A Gaussian process is fully describe by its mean μ and covariance kernel K , defined as

$$\mu = [\mu_0, \dots, \mu_N], K = [K(t_i, t_j)]_{i,j, 0 \leq i, j \leq N} \quad (5)$$

A specific structured kernel can be generated by linear time-varying stochastic differential equation (LTV-SDE)

$$\dot{\xi}(t) = A(t)\xi(t) + u(t) + F(t)w(t) \quad (6)$$

where $u(t)$ is the known system control input, $A(t)$ and $F(t)$ are time varying matrices of the system, and $w(t)$ is itself a zero-mean GP $w(t) \sim \mathcal{GP}(0, Q_C \delta(t - t'))$. Q_C denotes the power-spectral density matrix.

2.4 Factor Graph

Dellaert et al. [2017] A factor graph $G = \{\mathcal{U}, \mathcal{V}, \mathcal{E}\}$ is a bipartite graph, factor nodes $\phi_i \in \mathcal{U}$ and variable nodes $x_j \in \mathcal{V}$. Edges e_{ij} are always between factor nodes and variable nodes. We write X_i for an assignment to this set and the set of variable nodes adjacent to a factor ϕ_i is written as $\mathcal{N}(\phi_i)$. A factor graph F allows us to specify a joint density as a product of factors by defining the factorization of a global function $\phi(X)$ as

$$\phi(X) = \prod_i \phi_i(X_i) \quad (7)$$

Note that the independence relationships are encoded by the edges e_{ij} of the factor graph, with each factor ϕ_i a function of only the variables X_i in its adjacency set $\mathcal{N}(\phi_i)$.

71 3 RKHSs

72 A set of trajectories can be treated as a Hilbert space by defining vector-space operations of trajectories
 73 such as addition and scalar multiplication. Moreover, it can be upgraded to a vector-valued RKHS
 74 $\mathcal{H}$ by assuming the functional $\xi \mapsto \langle \xi(t), q \rangle$ is continuous for arbitrary $q \in \mathcal{C}$ and $t \in [0, 1]$. The
 75 trajectory norm in the RKHS measures the size of the local perturbations, and the correlation among
 76 them quantifies the complexity of the trajectory. The RKHS representation of trajectories allows
 77 efficient trajectory optimization through functional gradient descent.

78 3.1 RKHS Formulation

79 Assume our configuration space $\mathcal{C}$ is d dimension, the trajectories form an RKHS of vector-valued
 80 functions, and the associated reproducing kernel becomes a matrix valued kernel $K : [0, 1] \times [0, 1] \mapsto$
 81 $\mathcal{C} \times \mathcal{C}$.

$$K(t, t') = \begin{bmatrix} k_{1,1}(t, t') & k_{1,2}(t, t') & \dots & k_{1,d}(t, t') \\ k_{2,1}(t, t') & k_{2,2}(t, t') & \dots & k_{2,d}(t, t') \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ k_{d,1}(t, t') & k_{d,2}(t, t') & \dots & k_{d,d}(t, t') \end{bmatrix} \quad (8)$$

82 Eq. 8 defines the kernel matrix of joint interaction between timesteps t and t' . It has a intuitive
 83 physical interpretation as an inertial matrix of a rigid body changing over time. $k_{i,j}(t, t')$ represents
 84 the correlation between two configurations, i.e. how $\xi_i(t)$ affect $\xi_j(t')$. Overall, the matrix represents
 85 that perturbations of given joints propagate through time as well as rest of the joints. Since we have a
 86 vector-valued RKHS, for all $y \in \mathcal{C}$, equation 1 becomes

$$y^T \xi(t) = \langle \xi, K(t, \cdot) y \rangle_{\mathcal{H}} \quad (9)$$

87 Since a trajectory in RKHS admits a representation in terms of finite support $\{t_i\}_{i=1}^N \in \mathcal{T}$, we can
 88 write ξ as

$$y^T \xi(\cdot) = \sum_{i=1}^N a_i K(\cdot, t_i) y \quad (10)$$

89 where a_i is a coefficient vector. Consider a special case where the configuration vector $y = e_k$
 90 is the indicator of joint k , the inner product in $\mathcal{H}$ of functions $\xi^1(\cdot) = \sum_{i,k} a_{i,k} k(t_i, \cdot) e_k$ and
 91 $\xi^2(\cdot) = \sum_{j,k} b_{j,k} k(t_j, \cdot) e_k$ is defined as

$$\langle \xi^1, \xi^2 \rangle_{\mathcal{H}} = \sum_k \langle \xi_k^1, \xi_k^2 \rangle_{\mathcal{H}} = \sum_k \sum_{i,j} a_{i,k} b_{j,k} k(t_i, t_j) \quad (11)$$

92

$$\|\xi\|_{\mathcal{H}} = \sum_k \langle \xi, \xi \rangle_{\mathcal{H}} = \sum_k \sum_{i,j} a_{i,k} a_{j,k} k(t_i, t_j) \quad (12)$$

93 One kernel example is Gaussian kernel $k(t, t') = \exp(\|t - t'\|^2) / 2\sigma^2$ for some smoothing bandwidth
 94 σ . Then a trajectory is a weighted sum of radial basiss functions.

$$\xi(t) = \sum_{i,k} a_{i,k} \exp(\|t - t_i\|^2) / 2\sigma^2 e_k \quad (13)$$

95 where the coefficients $a_{i,k}$ assess how important a particular joint k at time t_i is to the
 96 overall trajectory. As a linear smoother, RKHS represents a trajectory as a weighted sum of local
 97 perturbations to the motions of different joints centered at different times.

98 3.2 Functional Gradient Descent

99 We define a cost function $\mathcal{H} \mapsto \mathbb{R}$ for trajectory optimization, which balances between a regularization
 100 term in size of the trajectory and the proximity to obstacles:

$$\mathcal{U}[\xi] = \mathcal{U}_{obs}[\xi] + \frac{\beta}{2} \|\xi\|_{\mathcal{H}}^2 \quad (14)$$

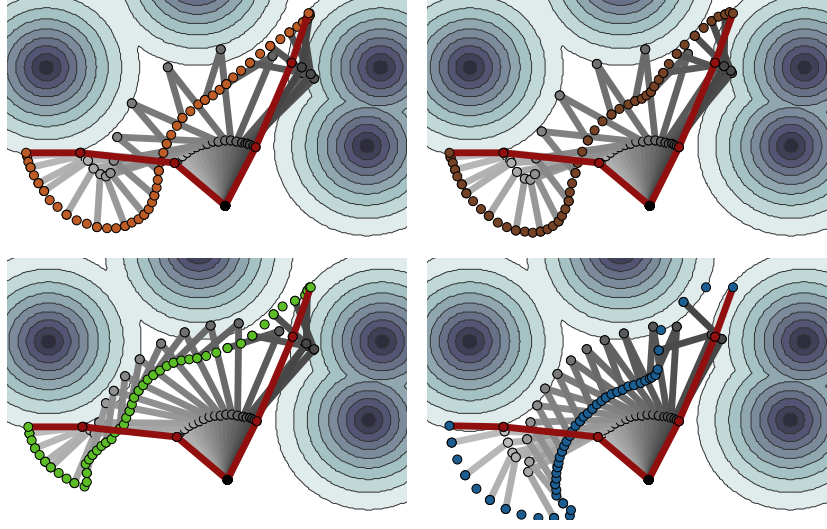


Figure 1: Marinho et al. [2016] They use 100 different random obstacle placements and fix start and goal configuration as experiment setups. This is a 3DoF Robot in C-space. Trajectory after 10 iterations: top-left: Gaussian RBF kernel, top-right: B-splines kernel, bottom-left: Laplaceian RBF kernel, bottom-right: Waypoints

101 We can then derive the functional gradient update at step n within an ellipsoid trust region about ξ^n
 102 in the RKHS:

$$\xi^{n+1} = \underset{\xi}{\operatorname{argmin}} \langle \xi - \xi^n, \nabla \mathcal{U}[\xi^n] \rangle_{\mathcal{H}} + \frac{\lambda}{2} \|\xi - \xi^n\|_{\mathcal{H}}^2 \quad (15)$$

103 The closed form solution to the minimization above can be computed:

$$\xi^{n+1}(\cdot) = \left(1 - \frac{1}{\lambda}\right) \xi^n(\cdot) - \frac{1}{\lambda} \nabla \mathcal{U}_{obs}[\xi^n](\cdot) \quad (16)$$

104 The set of time points $\mathcal{T}$ that support the trajectory contribute to the design of the kernel metric.

$$\|\xi\|_{\mathcal{H}}^2 = a^T K(\mathcal{T}, \mathcal{T}) a \quad (17)$$

105 Here a is the concatenation of all coefficients a_i over $\mathcal{T}$. It can be interpreted as a tensor metrix in
 106 Riemannian manifold and its inverse is the key element that bridges the functional gradient in RKHS
 107 and the functional gradient in Euclidean space

$$\nabla \mathcal{U} = K^{-1}(\mathcal{T}, \mathcal{T}) \nabla_E \mathcal{U} \quad (18)$$

108 They conduct an ablation experiment on kernel function, shown in Fig. 1. It's observed that Waypoints
 109 and Laplacian RBF kernels with large widths have similar behavior, while Gaussian RBF and B-
 110 splines kernels provide a smooth parameterization that allows the algorithm to take larger steps at
 111 each iteration.

112 4 GPs

113 To evoke the power of GPs, motion planning can be viewed as probabilistic inference. we seek to
 114 find a trajectory parameterized by ξ given desired events e . In this work, the desired event is collision
 115 free trajectory. The posterior density of ξ given e can be computed by Bayes rule from a prior and
 116 likelihood

$$p(\xi|e) \propto p(\xi)p(e|\xi) \quad (19)$$

117 where $p(\xi)$ is the prior on ξ that encourages smooth trajectories, and $p(e|\xi)$ is the likelihood that the
 118 trajectory ξ is collision free. The optimal trajectory ξ is then found by the maximum a posteriori
 119 (MAP) estimator. With a further formulation of inference as least squares optimization, they can
 120 perform inference very efficiently, so motion planning is extremely fast.

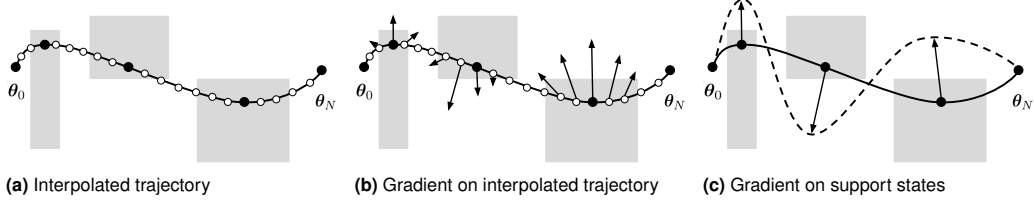


Figure 2: Mukadam et al. [2016] An example showing how GP interpolation is used during optimization. (a) shows the current iteration of the trajectory (black curve) parameterized by a sparse set of support states (black circles) and interpolated states (white circles) by GP regression. Then, in (b) cost is evaluated on all states and their gradients are illustrated by the arrows. Finally, in (c) the cost and gradient information is propagated to just the support states illustrated by the larger arrows such that only the support states are updated that parameterize the new trajectory (dotted black curve).

4.1 Gaussian Interpolation

Mukadam et al. [2016] The solution to the initial problem of the LTV-SDE in Eq. 6 is

$$\xi(t) = \Phi(t, t_0)\xi_0 + \int_{t_0}^t \Phi(t, s)(u(s) + F(s)w(s))ds \quad (20)$$

where $\Phi(t, s)$ is the state transition matrix that transfers state from time s to time t . The mean μ and covariance function K of the GP defined by this LTV-SDE conditioned on an observation goal state (fixed both endpoints) can be solved in closed form in Mukadam et al. [2016]. This particular construction of the prior leads to a Gauss-Markov model that generates a GP with exactly sparse tridiagonal precision matrix

$$K^{-1} = B^T Q^{-1} B \quad (21)$$

where block diagonal matrices B, Q are defined in Mukadam et al. [2016]. With this inverse kernel, they can similarly define cost functional as a tradeoff between smoothness and obstacle costs, and compute functional gradient with respect to ξ . The main advantage of GPs is that they can be parametrized by only a sparse set of support states but the trajectory can be queried at any time of interest through Gaussian interpolation, as shown in Fig. 2.

4.2 Factor Graph Inference

Mukadam et al. [2018] Given the Markovian structure of the trajectory ξ and sparsity of inverse kernel matrix K^{-1} , the posterior distribution $p(\xi|e)$ can be further factored such that MAP inference can be equivalently viewed as performing inference on a factor graph. Similar to Eq. 7, the posterior can be factored as

$$p(\xi|e) \propto \prod_i f_i(\Xi_i) \quad (22)$$

where f_i are factors on variable subsets Ξ_i . Specifically, we factor the prior

$$p(\xi) \propto f_0^p(\xi_0) f_N^p(\xi_N) \prod_{i=1}^N f_i^{gp}(\xi_i, \xi_{i+1}) \quad (23)$$

Here ξ_i represents the discretized i th configuration and $f_0^p(\xi_0)$ and $f_N^p(\xi_N)$ define the prior factor on start and goal states,

$$f_i^p(\theta_i) = \exp(-\frac{1}{2} \|\xi_i - \mu_i\|_{K_i}^2), i = 0, N \quad (24)$$

The GP prior factor is

$$f_i^{gp}(\xi_i, \xi_{i+1}) = \exp(-\frac{1}{2} \|\Phi(t_{i+1}, t_i)\xi_i - \xi_{i+1} + u_{i,i+1}\|_{Q_{i,i+1}}^2) \quad (25)$$

where $u_{a,b} = \int_{t_a}^{t_b} \Phi(b, s)u(s)ds$, Φ is the state transition matrix and $u(s), Q_{i,i+1}$ are defined in Mukadam et al. [2016]. There exist two types of obstacle factors: regular factors f_i^{obs} and interpolated obstacle factors $f_{\tau_j}^{intp}$. The likelihood $l(\xi; e)$ is the product of all obstacle factors

$$l(\theta; e) = \prod_{i=1} [f_i^{obs}(\theta_i) \prod_{j=1} f_{\tau_j}^{intp}(\theta_i, \theta_{i+1})] \quad (26)$$

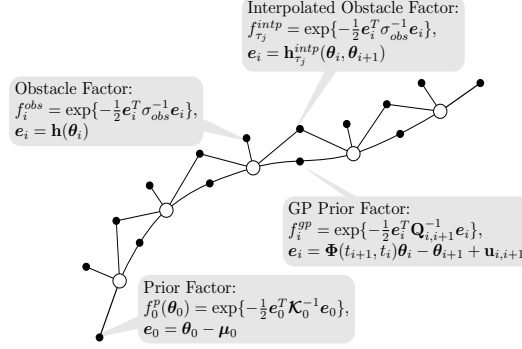


Figure 3: Mukadam et al. [2018] A factor graph of an example trajectory optimization problem showing support states (white circles) and four kinds of factors (black dots), namely prior factors on start and goal states, GP prior factors that connect consecutive support states, obstacle factors on each state, and interpolated obstacle factors between consecutive support states.

145 The regular obstacle factor describes the obstacle costs on a single state ξ_i , defined as

$$f_i^{obs} = \exp(-\frac{1}{2} \|h(\xi_i)\|_{\sigma_{obs}}^2) \quad (27)$$

146 where $h(\xi_i)$ is vector-valued obstacle cost function and σ_{obs} is a hyperparameter matrix. The
 147 interpolated obstacle factors are obtained through fast interpolation in the Gauss-Markov Model at
 148 time τ_j satisfying $t_i \leq \tau_j \leq t_{i+1}$:

$$\begin{aligned} f_{\tau_j}^{intp}(\theta_i, \theta_{i+1}) &= \exp(-\frac{1}{2} \|h(\xi_{\tau_j})\|_{\sigma_{obs}}^2) \\ &= \exp(-\frac{1}{2} \|h_{\tau_j}^{intp}(\xi_i, \xi_{i+1})\|_{\sigma_{obs}}^2) \end{aligned} \quad (28)$$

149 An example factor graph that combines all the above factors is shown in Fig. 3. Now to solve the
 150 MAP inference in Eq. 19, we perform a minimization on the negative log of the posterior distribution

$$\begin{aligned} \xi^* &= \underset{\xi}{\operatorname{argmax}} p(\xi) l(\xi; e) \\ &= \underset{\xi}{\operatorname{argmax}} (-\log(p(\xi) l(\xi; e))) \\ &= \underset{\xi}{\operatorname{argmax}} (\frac{1}{2} \|\xi - \mu\|_K^2 + \frac{1}{2} \|h(\xi)\|_{\Sigma_{obs}}^2) \end{aligned} \quad (29)$$

151 where Σ_{obs} is the stacked obstacle hyperparameter matrix. Now the construction becomes a nonlinear
 152 least square problem, so iterative approaches such as Gauss-Newton can be used to solve the equation.
 153 Linearizing the cost function around the trajectory ξ ,

$$h(\xi + d\xi) \approx h(\xi) + Hd\xi \quad (30)$$

154 where H is the jacobian matrix of $h(\xi)$. Then the equation 29 becomes

$$\delta\xi^* = \underset{\delta\xi}{\operatorname{argmax}} (\frac{1}{2} \|\xi + \delta\xi - \mu\|_K^2 + \frac{1}{2} \|h(\xi) + H\delta\xi\|_{\Sigma_{obs}}^2) \quad (31)$$

155 The optimal update $\delta\xi^*$ results from solving the linear system

$$(K^{-1} + H^T \Sigma_{obs}^{-1} H) \delta\xi^* = -K^{-1}(\xi - \mu) - H^T \Sigma_{obs}^{-1} h(\xi) \quad (32)$$

156 Once the linear system is solved, the update rule

$$\xi = \xi + \delta\xi^* \quad (33)$$

157 is applied until convergence.

5 Discussion and Conclusion

In this summary we have seen Marinho et al. [2016] and Mukadam et al. [2018] that respectively formulate continuous trajectory optimization as RKHSs and GPs and reach efficient performance in practice. Their methods provide a natural solution to avoid the fine discretization requirement of previous optimization methods, which is the kernel functions that allow interpolation through fewer support states. While RKHS representation permits the larger gradient step than less-flexible representations, GPMP2 dramatically improves planning speed through inference on factor graph and solving least square problem efficiently.

References

- Frank Dellaert, Michael Kaess, et al. Factor graphs for robot perception. *Foundations and Trends® in Robotics*, 6(1-2):1–139, 2017.
- Anca D Dragan, Nathan D Ratliff, and Siddhartha S Srinivasa. Manipulation planning with goal sets using constrained trajectory optimization. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4582–4588, 2011.
- Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, and Stefan Schaal. Stomp: Stochastic trajectory optimization for motion planning. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4569–4574, 2011.
- Z Marinho, A Dragan, A Byravan, B Boots, G Gordon, and S Srinivasa. Functional gradient motion planning in reproducing kernel hilbert spaces. *arXiv preprint arXiv:1601.03648*, 2016.
- Mustafa Mukadam, Xinyan Yan, and Byron Boots. Gaussian process motion planning. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 9–15. IEEE, 2016.
- Mustafa Mukadam, Jing Dong, Xinyan Yan, Frank Dellaert, and Byron Boots. Continuous-time gaussian process motion planning via probabilistic inference. *The International Journal of Robotics Research (IJRR)*, 37(11):1319–1340, 2018.
- Nathan Ratliff, Matt Zucker, J Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Gradient optimization techniques for efficient motion planning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- John Schulman, Yan Duan, Jonathan Ho, Alex Lee, Ibrahim Awwal, Henry Bradlow, Jia Pan, Sachin Patil, Ken Goldberg, and Pieter Abbeel. Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research (IJRR)*, 33(9): 1251–1270, 2014.